UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
Campus Medianeira

FABIANA LORENSONI
ISABELLA VIEIRA JOENCK

INTEGRAÇÃO DO PROLOG COM PYTHON E JAVASCRIPT:
TUTORIAL DE APLICAÇÕES PRÁTICAS

MEDIANEIRA
2026

1 INTRODUÇÃO

A programação lógica constitui um paradigma de programação fundamentado na representação de conhecimentos por meio de fatos, regras e consultas. O Prolog é uma linguagem representativa desse paradigma e pode ser utilizado em conjunto com linguagens de propósito geral, que ficam responsáveis pela entrada e apresentação dos dados.

Este tutorial apresenta duas integrações práticas: Python com Prolog, para verificar a aprovação de alunos a partir de notas, e JavaScript com Prolog, para identificar números pares e ímpares. O objetivo é permitir que o leitor reproduza os exemplos, compreenda a responsabilidade de cada linguagem e verifique o funcionamento das consultas.

2 REFERENCIAL TEÓRICO

2.1 Programação lógica e Prolog

No Prolog, fatos representam informações conhecidas e regras descrevem condições para que uma relação seja verdadeira. Uma consulta solicita ao interpretador a verificação de uma condição. O mecanismo de inferência busca satisfazer a consulta a partir das definições disponíveis (SWI-PROLOG, 2026).

2.2 Integração entre linguagens

Integrar linguagens permite combinar recursos distintos em uma mesma aplicação. No primeiro exercício, a biblioteca PySwip fornece uma interface entre Python e SWI-Prolog. No segundo, o Node.js inicia o executável do SWI-Prolog como processo externo e interpreta seu código de saída. As duas estratégias efetivamente executam regras Prolog, mas usam mecanismos de comunicação diferentes (PYSWIP, 2026; NODE.JS, 2026).

3 DESENVOLVIMENTO

3.1 Exercício 1 — Sistema de aprovação de alunos

O programa solicita uma nota entre zero e dez. A regra de aprovação considera aprovado o aluno com nota maior ou igual a seis; caso contrário, considera-o reprovado. A interface e a validação da entrada são feitas em Python, enquanto a decisão é consultada no Prolog.

3.1.1 Ferramentas e estrutura dos arquivos

São necessários Python 3, SWI-Prolog e a biblioteca PySwip. Crie uma pasta denominada Exercicio_1_Python contendo os arquivos regras.pl e main.py. No terminal, instale a dependência com o comando python -m pip install pyswip. A instalação do SWI-Prolog precisa estar acessível à biblioteca.

3.1.2 Arquivo regras.pl

```
aprovado(Nota) :-
Nota >= 6.
reprovado(Nota) :-
Nota < 6.
A condição Nota >= 6 define a aprovação. A regra complementar identifica
```

notas inferiores a seis. Os valores consultados devem ser numéricos.

3.1.3 Arquivo main.py

```
from pathlib import Path
from pyswip import Prolog
prolog = Prolog()
arquivo = Path(__file__).resolve().parent / "regras.pl"
prolog.consult(str(arquivo))
print("=== SISTEMA DE APROVACAO ===")
try:
nota = float(input("Digite a nota: ").replace(",", "."))
if nota < 0 or nota > 10:
print("Nota invalida!")
elif list(prolog.query(f"aprovado({nota})")):
print("Aluno APROVADO!")
else:
print("Aluno REPROVADO!")
except ValueError:
print("Digite uma nota valida!")
```

3.1.4 Execução e resultados

Abra o terminal na pasta Exercicio_1_Python e execute python main.py. Digite uma nota quando solicitado. Os exemplos a seguir indicam as saídas esperadas; o grupo informou ter executado a integração com sucesso.

```
Digite a nota: 8
Aluno APROVADO!
Digite a nota: 4
Aluno REPROVADO!
```

3.2 Exercício 2 — Verificador de números pares e ímpares

O programa recebe um número inteiro por meio do JavaScript e utiliza uma regra em Prolog para identificar se ele é par. O Node.js invoca o SWI-Prolog como processo externo, de modo que a lógica da classificação permanece no arquivo de regras.

3.2.1 Ferramentas e estrutura dos arquivos

São necessários Node.js e SWI-Prolog. Crie a pasta Exercicio_2_JavaScript e, dentro dela, os arquivos regras.pl e main.js. Este exemplo utiliza apenas módulos nativos do Node.js e não requer instalar pacotes adicionais com npm. No Windows, confirme o caminho do executável swipl.exe e ajuste a constante swipl caso a instalação esteja em outra pasta.

3.2.2 Arquivo regras.pl

```
par(N) :-
integer(N),
0 is N mod 2.
impar(N) :-
integer(N),
```

1 is abs(N mod 2).

O operador mod calcula o resto da divisão inteira. O número é par quando esse resto é zero. O uso de abs na regra de ímpar também permite classificar corretamente inteiros negativos.

3.2.3 Arquivo main.js

```
const { spawnSync } = require("child_process");
const path = require("path");
const readline = require("readline");
const swipl = "C:\Program Files\swipl\bin\swipl.exe";
const regras = path.join(__dirname, "regras.pl");
const rl = readline.createInterface({
input: process.stdin,
output: process.stdout
});
console.log("=== VERIFICADOR DE PAR OU IMPAR ===");
rl.question("Digite um numero inteiro: ", (entrada) => {
const numero = Number(entrada.trim());
if (entrada.trim() === "" || !Number.isSafeInteger(numero)) {
console.log("Erro: digite um numero inteiro valido!");
rl.close();
return;
}
const consulta = `par(${numero})`;
const resultado = spawnSync(swipl,
["-q", "-s", regras, "-g", consulta, "-t", "halt"],
{ encoding: "utf8" });
if (resultado.error) {
console.log("Erro ao iniciar o Prolog:");
console.log(resultado.error.message);
} else if (resultado.status === 0) {
console.log(`O numero ${numero} e PAR!`);
} else if (resultado.status === 1 && !resultado.stderr) {
console.log(`O numero ${numero} e IMPAR!`);
} else {
console.log("Erro ao consultar o Prolog:");
console.log(resultado.stderr || `Codigo: ${resultado.status}`);
}
rl.close();
});
```

3.2.4 Execução e resultados

Abra o terminal na pasta Exercicio_2_JavaScript e execute node main.js. Informe um número inteiro. O programa consulta a regra par(N) e interpreta o retorno do processo Prolog: saída zero indica consulta bem-sucedida; saída um, sem erro, indica consulta falsa. O grupo informou ter testado números pares e ímpares com sucesso.

```
Digite um numero inteiro: 10
O numero 10 e PAR!
Digite um numero inteiro: 7
O numero 7 e IMPAR!
```

4 COMPARAÇÃO DAS INTEGRAÇÕES

Na aplicação Python, a comunicação é realizada pela biblioteca PySwip, que carrega diretamente o arquivo de regras e consulta os predicados. Na aplicação JavaScript, a comunicação ocorre pela execução de um processo SWI-Prolog com argumentos de linha de comando. Em ambos os casos, a lógica de decisão é mantida no Prolog, enquanto a outra linguagem gerencia a interação com o usuário.

Como cuidados de implementação, é necessário validar os dados antes das consultas e garantir que os arquivos de regras estejam no local esperado. No exemplo JavaScript, o caminho do executável pode precisar ser adaptado ao computador utilizado. Para a entrega, recomenda-se incluir capturas reais dos testes feitos pelo grupo nas respectivas pastas de código.

5 CONCLUSÃO

Os exercícios demonstram que a programação lógica pode ser combinada com linguagens de uso geral em aplicações simples. A integração com Python utiliza uma biblioteca específica, enquanto a integração com JavaScript utiliza um processo externo. As duas abordagens permitem separar a apresentação dos dados da avaliação das regras lógicas.

A realização das atividades contribui para compreender fatos, regras e consultas no Prolog, bem como os mecanismos de comunicação entre programas. O tutorial reúne os procedimentos necessários para reproduzir os exemplos e serve de apoio à apresentação prática do grupo.

REFERÊNCIAS

NODE.JS. Child process: Node.js documentation. Disponível em: https://nodejs.org/api/child_process.html. Acesso em: 27 out. 2026.

PYSWIP. PySwip documentation. Disponível em: https://pyswip.readthedocs.io/. Acesso em: 27 out. 2026.

SWI-PROLOG. SWI-Prolog documentation. Disponível em: https://www.swi-prolog.org/pldoc/. Acesso em: 27 out. 2026.

PYTHON SOFTWARE FOUNDATION. Python documentation. Disponível em: https://docs.python.org/3/. Acesso em: 27 out. 2026.